Working with Codex

In crew 1.0, the interactive Claude session implements and Codex reviews. Both agents read the same instructions, and the same context hook can run under both. This guide explains how to set up the Codex half and what is proven about it so far.

What Each Side Reads

Surface	Claude Code	Codex
`AGENTS.md` (80 lines or fewer, generated)	through `@AGENTS.md` in `CLAUDE.md`	natively
`.claude/rules/<subsystem>.md` (`paths:`-scoped)	yes, when a matching file is read	**no** (Codex has no path-scoped rules)
crew's context hook	`plugins` hooks.json	`.codex/hooks.json` (generated)
`CLAUDE.md`	natively	fallback via `project_doc_fallback_filenames`

Codex has no path-scoped rules, so its hook's `PostToolUse` slice is the only per-file channel.

Generate the Files

Run these from the repository root. Each command writes nothing if the files are already current. With `--check`, it writes nothing at all and exits 1 on drift.

```
S="<crew plugin root>/hooks/scripts"

python3 "$S/crew_instructions.py" agents      # AGENTS.md; only the crew:keep block is yours

python3 "$S/crew_instructions.py" rules       # .claude/rules/<subsystem>.md from .crew/codemap/

python3 "$S/crew_instructions.py" codex       # .codex/hooks.json and .codex/config.toml
```

```
python3 "$S/crew_instructions.py" codex --check
```

- `.codex/hooks.json` is built from the same hook table as crew's Claude registration (`HOOK_TABLE` in `crew_instructions.py`). Each entry has a `command` (bash) and a `commandWindows` (PowerShell, `-Harness codex`), both running the `crew-hook` launcher for the event, which dispatches to `crew-context` only. The paths inside are absolute and belong to one machine, so regenerate the file on each machine rather than committing it.
- `.codex/config.toml` sets `project_doc_fallback_filenames = ["CLAUDE.md"]` only. **No `[profiles.*]` table** -- MEASURED on codex-cli 0.154.0: `profiles` is on Codex's own project-local config denylist and is stripped from this file every time it loads, trusted or not, so a `review`/`work` profile defined here would never apply. Codex reads project config only for a **trusted** project, and only that project-config subset survives the strip.

Review and Work

There are no project-scoped `review`/`work` profiles to select with `--profile` -- pass the settings explicitly instead:

```
codex exec --json --sandbox read-only "<review prompt>"   # the reviewer
(review_run.py's command_for)

codex exec --sandbox workspace-write "<task>"              # only if you choose
Codex to implement
```

To check that Codex answers before you rely on it, run the probe from anywhere:

```
python3 "${CLAUDE_PLUGIN_ROOT}/hooks/scripts/provider_probe.py" codex --root <repo>
```

It makes one real call with the reviewer's own command line (`review_run.command_for`, which adds `--skip-git-repo-check` and `-C <repo>`) and runs it with the repo root as its working directory, so it does not hit "Not inside a trusted directory" the way a `codex exec` typed in a `/tmp` export does. `codex: ok` is exit 0; `codex: FAILED - <reason>` (a failed call, an incomplete event stream, or a 120 s timeout) is exit 1; `codex: not installed` is exit 2.

If you want named `review`/`work` profiles for convenience, they are a **user-level** concept, not a project one: `codex exec --help` (codex-cli 0.155.1) documents `--profile NAME` as layering

`$CODEX_HOME/NAME.config.toml` on top of the base user config -- a file under `$CODEX_HOME`, never this project's `.codex/config.toml`. Create `$CODEX_HOME/review.config.toml` / `$CODEX_HOME/work.config.toml` yourself and pass `--profile review`/`--profile work` against those.

A review that exits non-zero, prints nothing, times out or ends in a failed turn is INCOMPLETE, never CLEAN. That is a tool failure, and the round is refunded automatically, up to three per ticket (for example when Codex answers with its usage-limit `error` event). A refunded round that was not a usage limit or a timeout is retried once by `review_run.py` itself, after a 30-second backoff; a `review: retry:` line says whether it was, and why not. What crew relies on from Codex's `--json` stream is in `plugin/crew/docs/external-tool-formats.md`.

The review prompt Codex reads ends with a **development standards checklist**: the rules and self-check questions of every standard that applies to the change (crew's generic set, any matching per-language set, and the repository's `.crew/standards.md` overlay). The author's own self-check answers are withheld from the prompt, so Codex judges each standard's applicability itself, and the prompt says the list does not bound the review: a defect outside it is reported the same way. Withheld from the prompt is not hidden: the answers stay in `.work/tickets/<id>/selfcheck.md`, which Codex, with read access to the checkout, could open; the prompt never names that file. For a ticket with an approval receipt (or one whose receipt cannot be proven absent), `review_run.py` will not launch Codex, or reserve a round, until the author's self-check is complete and stamped for the exact bundle Codex is about to read. A ticket with no approval receipt is told the gate does not apply, and during a declared incident (`/crew:emergency`) the gate stands down and logs the skip. The self-check is asked for last: a CLEAN receipt that already covers the bundle answers CLEAN without it, and a tree the verify gate has not passed is refused (exit 9) before it.

After the standards checklist comes the **recurring-findings checklist**: the defect classes earlier reviews kept finding (processes and races, claims not true at the commit, tests that cannot fail, fail-open handling, PowerShell/Bash drift, guard bypass, version and registration), only those whose path globs meet the files the diff changes, a few probes each. It is the same list `/crew:implement` showed the developer before the first plan step. It does not bound the review, and an item the diff does not touch is not a finding. When the bundle's manifest cannot say which files changed, every class is listed under an `UNKNOWN:` line rather than none. Each class names the development standards it is a concrete instance of (its `seen:` line, for example GEN-09): the list is derived from the standards, a test holds every id it names to one that exists, and where a probe and its standard disagree the standard wins.

Between the verify gate and the self-check, `review_run.py` lints the bundle's changed files with the linters `.crew/verify.json` lists under `preReview` (L-0574). Each file is linted at its base and at the bundle, and a finding the bundle adds refuses the round (exit 9, nothing spent) before Codex is launched. `--allow-unverified` does not override a new finding. A linter that could not run, or could not parse a changed file, reads `COULD NOT CHECK`, never a pass. It refuses too, unless `--allow-unverified` is given, and `review.json` records that override as `prereview.overridden`. When the verify gate has not passed the tree, the prompt's test receipts keep their `MISSING` and `Gate answer for HEAD` lines and add one line saying such a round runs only under `--allow-unverified`, recorded as `gate.overridden` in `review.json`, that `/crew:done` still needs a clean gate, and that the missing pass alone is that recorded override, not a defect (T-0101).

Once the clone's merge train is armed (`crew_train.py arm`), the gate round takes it before any of that linting (L-0526): after the CLEAN receipt and the verify gate, `review_run.py` calls `crew_train.acquire`, and a ticket that is waiting behind an overlapping one, must merge its base first, or meets a train it cannot read is refused with exit 10, nothing spent and Codex not launched. An unarmed clone never asks. The prompt also carries a `== Catch-up merges (rerere) ==` block listing every file a catch-up merge replayed from an earlier rerere resolution, so Codex reviews each as a change; a merge log it cannot read is written there as `UNREADABLE`.

When a Pinned Codex Model Is Retired

A pin to a model Codex no longer serves falls back to `qa.fallback`, and since L-0712 the fallback runs on the provider that serves its family: a `gpt-6.1-sol` fallback dispatches to Codex with that model, not to Claude. A fallback in the author's family (the shipped `claude-sonnet-5`, on Claude-written work) is skipped for the next provider in `qa.order` that did not write the diff and answers, such as Kimi. When none answers, the round is INCOMPLETE and refunded rather than spent on a same-family read. `/crew:status` prints the share of same-family rounds across every ticket.

When Codex Hits a Usage Limit

Having `codex` on `PATH` does not mean it can review: a logged-out, rate-limited or out-of-credits Codex is installed and fails at the first call. So `/crew:review` makes one minimal real call first, before it reserves a round, with the round's own model and effort:

```
python3 "$S/review_run.py" --root . --ticket <id> --scratch <dir> --provider codex \
  --model <model> --effort <effort> --probe
```

It prints `PROBE=<outcome>` and `PROBE_DETAIL=<the answer or the quoted error>` and exits `0` (ok), `5` (limited), `6` (failed) or `7` (unknown: no answer within 120 s). Nothing is reserved either way. `limited` means the call failed and its error matched one of Codex's own limit messages (usage limit, out of credits, spend cap, rate limit, quota, a plan without Codex, or a retry limit on HTTP 429), listed with their `error.rs` lines in `hooks/scripts/review_limit.py`. On `limited` Codex is skipped - even when `qa.provider` pins `codex` - with the error quoted, and the round goes to the next cross-family provider. When none is left, no reviewer runs: the outcome is recorded INCOMPLETE and refunded (`review_ledger.py --no-reviewer --reason <why>`), spending no round. A Claude read after a limit runs only when you ask for it (`--same-family "codex limit"`, crew 1.2.6), announced as `same-family (codex limit)` and labelled in the ledger, because it is not an independent review.

A limit hit in the middle of a round leaves that round INCOMPLETE and prints `review: codex usage limit in round N: ...`. It is recorded in `<git-common-dir>/crew/review-limit/<ticket>.json`, so the next `/crew:review` answers `limited` from the record without calling Codex again. Once that next round is reserved the record stops applying, and the round after it probes Codex live. The record is written after the round's verdict: when it cannot be written the round still ends INCOMPLETE (exit 3), the line says `could not record it (...)`, and the next probe calls Codex live instead.

What Is Proven, and What Is Only Configured

Run the probe. The sample below shows the probe's format with this build's values. The `hooks feature` value was measured with `codex features list`. The probe itself has only been run against a stub `codex` in the test suite (`tests/test_crew_instructions.py`).

```
python3 "$S/crew_instructions.py" codex-probe

codex: /usr/local/bin/codex (codex-cli 0.155.1)

hooks feature: enabled

project files: current
```

```
PostToolUse: configured, not proven

SessionStart: configured, not proven

SubagentStart: configured, not proven

UserPromptSubmit: configured, not proven
```

- **Established by reading the codex 0.155.1 binary, not its docs:** the hook events (`SessionStart`, `UserPromptSubmit`, `PostToolUse`, `SubagentStart` and others), the handler keys `command`, `commandWindows`, `timeout` and `additionalContextLimit`, and a `SubagentStartHookSpecificOutputWire` output type. `codex features list` reports `hooks stable true`.
- **Not measured:** whether a hook's `additionalContext` reaches the Codex model, and whether Codex honours `profiles` from a project's `.codex/config.toml`. A live test run would need Codex to write its session state under `~/.codex`, and this build did not do that.
- The probe changes an event's line from "configured, not proven" to "hook invoked Nx under Codex" only when the context log records an invocation under the `codex` harness. Even then it claims only that the hook ran, not that the model received the context. To prove delivery, rerun the seeded-note test from Confirming recall reaches your sessions with `codex exec` in place of `claude -p`.

When Codex Is Barred

A review is independent only when it comes from a different model family than the author. If Codex wrote the diff, it cannot review that diff. The order falls through to Copilot, then to Claude's `reviewer`.